ChatGPT

Advancing Threat Modeling with Semantic Knowledge Graphs

White Paper by Dinis Cruz and ChatGPT Deep Research

Executive Summary

Threat modeling is a cornerstone of secure software design, yet today it remains largely a manual, subjective exercise that varies widely between practitioners. Most threat models are captured in static documents or diagrams, often siloed within individual teams and lacking a common format or language. These limitations hinder scalability and knowledge sharing, making it difficult to objectively compare or integrate threat models across an organization. This white paper presents a forward-looking vision for transforming threat modeling through **Semantic Knowledge Graphs**. By representing threat models as rich, interconnected graphs of data – complete with machine-understandable meaning (semantics) – we can elevate them to first-class artifacts that are shareable, queryable, and analyzable at scale.

In the proposed approach, threats, assets, mitigations, and contextual information (such as business impacts, compliance requirements, and real-world incidents) are represented as nodes and relationships in a knowledge graph. This enables **contextual enrichment** of threat models with data from multiple sources and ontologies, aligning technical details with business context in a unified model. Using semantic graphs, organizations can overlay multiple security frameworks (e.g. STRIDE, MITRE ATT&CK, OWASP Top 10) onto their systems and see how they interconnect – something not feasible with traditional static models. Furthermore, the graph representation allows the use of inference engines and queries to automatically identify risks and gaps, providing a more objective and repeatable analysis of threats.

We outline an architecture wherein threat modeling moves from a one-time paperwork exercise to a continuous, dynamic process. **LLMs (Large Language Models)** and automation assist in extracting knowledge and populating the graph, while a **memory-first graph database** (such as the open-source MGraph-DB) stores and serves the knowledge. This paper details how such a system can be built and integrated: from schema design and multi-ontology support, to graph construction pipelines and integration with development and security tools. We also address technical considerations like ensuring determinism, provenance of data, and performance in handling graph queries. By treating threat models as living knowledge graphs, security teams can maintain a clear chain of reasoning, visualize complex relationships, and keep models up-to-date as systems evolve.

Ultimately, semantic knowledge graphs promise to turn threat modeling into a **scalable, collaborative, and context-rich discipline**. The future outlook section explores how this approach paves the way for continuous threat modeling, real-time security insights during development, and industry-wide knowledge sharing of threat intelligence. This white paper is co-authored by Dinis Cruz and ChatGPT Deep Research, combining practical experimentation with cutting-edge AI research to chart a path forward for the threat modeling community.

Introduction

Threat modeling is the process of systematically identifying and assessing potential threats to a system's security. In practice, it often involves drawing data flow diagrams, enumerating threat scenarios (using methods like STRIDE or ATT&CK), and brainstorming mitigations. The goal is to anticipate "what could go wrong" and address design flaws before they manifest as vulnerabilities. However, performing threat modeling effectively across modern, complex systems has become increasingly challenging. Many organizations struggle to incorporate threat modeling into fast-paced development lifecycles, and those that do often produce static reports that quickly become outdated. Moreover, because threat modeling heavily relies on human expertise and judgment, the results can be **inconsistent and subjective**. As one expert quipped, threat modeling is essentially an attempt to make "a fairly subjective process more objective, repeatable & consistent" [1].

Several issues plague the current state of threat modeling. First, it tends to be siloed – security architects or consultants may perform threat modeling in isolation, with limited input from developers or business stakeholders. The outputs (spreadsheets, diagrams, Word documents) are not easily accessible or usable by others outside the immediate team. This means hard-won threat knowledge often isn't shared or reused across projects. Second, there is a lack of standardization in how threat models are represented. Organizations might follow different methodologies (STRIDE, PASTA, LINDDUN, etc.), each with its own terminology and focus, leading to confusion especially for teams without deep security expertise [2]. With numerous threat modeling frameworks available, teams can be unsure which to adopt and how to compare results, sometimes leading to "process saturation" and analysis paralysis [2]. Third, traditional threat modeling doesn't scale well. It's time-consuming to manually analyze every component of every new application or feature, so threat modeling is often done infrequently or only on high-profile projects. This leaves coverage gaps where many systems go un-modeled, or models are done once and not updated regularly. Finally, because threat modeling exercises are typically based on the knowledge of individuals in a room, they may miss emerging risks or broader context. For example, a threat model might overlook a newly discovered vulnerability or an evolving attack technique simply because the team wasn't aware of it at the time. Important context – such as real incident data, compliance requirements, or business priorities – may not be systematically integrated into each model, making it harder to assess which threats matter the most.

Together, these challenges paint a picture of a practice that, while valuable, has remained something of a craft or art form. The outputs are often point-in-time, non-interactive artifacts that don't readily plug into other security processes or decision-making tools. In an era of rapid DevOps and increasingly complex, distributed architectures, the need to modernize threat modeling is clear. We need approaches that can break down silos, inject up-to-date knowledge, and keep pace with change – without depending solely on scarce expert hours. In short, we need to **evolve threat modeling into a data-driven, collaborative, and continuous practice**.

One promising path to achieve this is by leveraging **semantic knowledge graphs**. By representing threat models in a graph format enriched with semantic context, we can address many of the aforementioned issues. This paper explores that path, showing how the marriage of threat modeling and semantic graphs (aided by AI) can yield a scalable and interoperable discipline. First, we provide an overview of what semantic knowledge graphs are and why they are well-suited to this domain.

Current Challenges in Threat Modeling

Before diving into the solution, it's worth summarizing the key hurdles that any improved approach must overcome:

- **Subjectivity and Inconsistency:** Traditional threat modeling relies on individual expertise. Different people might identify different threats for the same system, and even the same person might produce varying results on different days. There is often no single "source of truth" for threats, and it's difficult to verify completeness. As noted, the very aim of formalized threat modeling is to bring objectivity and consistency to what starts as a subjective exercise [1]. Yet, without robust frameworks or automation, subjectivity remains a problem.

- **Siloed Knowledge:** Threat models are usually documented in prose or diagrams that are not machine-readable or easily sharable. They often live in isolated documents or proprietary files. This makes it hard to aggregate knowledge or learn from past models. One team might solve a security design problem that another team later struggles with, simply because threat model insights aren't widely disseminated. There is no common knowledge base – each model is an island. Furthermore, because threat models are not typically integrated with DevOps tools, they can be overlooked during fast-moving development cycles.

- **Lack of Scalability:** Manually creating and maintaining threat models for every system, let alone every software update, is resource-intensive. Many organizations only threat-model critical projects or do one-off workshops, which doesn't scale to modern development practices. As systems grow into dozens of microservices and deployments per day, a manual approach simply cannot cover the continuously changing attack surface. Automation in mapping architectures and identifying threats has been minimal until recently. Consequently, threat modeling often lags behind development, instead of being a continuous guardrail.

- **Fragmented Methodologies:** With multiple threat modeling frameworks and notations, there's confusion and fragmentation. For example, some teams use STRIDE (focused on categories of threats), others use attack trees, others use kill chains or misuse cases. Each method brings a perspective, but without a unifying way to link them, organizations cannot easily combine or switch methods. Teams lacking a seasoned threat modeling expert find it difficult to judge which process to use and risk choosing an inadequate approach [3]. This fragmentation also means tool support is limited – a tool that supports one methodology might not support another, impeding a holistic view.

- **Static and Context-Poor Models:** Threat models are often snapshots of a system at a point in time. They might not incorporate the latest threat intelligence or incident data. For instance, if a major supply-chain attack was reported in the news, that insight might not find its way into existing threat models, especially if they are not regularly revisited. Likewise, compliance requirements or business impact information (e.g., which assets are "crown jewels") may reside in separate documents and aren't linked to the technical threat model. This lack of context can lead to mis-prioritization – e.g. treating all threats as equal, rather than focusing on those that could cause the most business damage or violate critical compliance controls.

In summary, traditional threat modeling can be **labor-intensive, ad-hoc, and siloed**, leading to gaps between intended and actual security coverage. Any next-generation approach should strive to be continuous, comprehensive, and collaborative – capturing expert knowledge in a reusable form and

augmenting it with real data wherever possible. This sets the stage for why semantic knowledge graphs are a compelling solution to explore.

Overview of Semantic Knowledge Graphs

A **semantic knowledge graph** is a way of representing information that emphasizes relationships and meaning. In a knowledge graph, entities (nodes) such as "Web Application", "Threat Actor", "Vulnerability X", or "Mitigation Y" are connected by relationships (edges) such as "attacks", "has vulnerability", "mitigated by", etc. What makes it "semantic" is that each node and relationship is defined in a schema or ontology that gives it well-defined meaning. In other words, the graph isn't just arbitrary data – it follows a model (often based on standards or ontologies) that both humans and machines can understand. This semantic layer enables advanced features like inference (deriving new facts from known ones) and interoperability (since common ontologies allow different systems to exchange data with shared understanding).

Semantic knowledge graphs have their roots in the Semantic Web movement, where technologies like RDF (Resource Description Framework) and OWL (Web Ontology Language) were created to make data on the web machine-readable and linked. In recent years, knowledge graphs have been adopted beyond academia, by industry giants for things like search (Google's Knowledge Graph) and personal assistants (to connect facts about entities). In the context of cybersecurity and threat modeling, a semantic knowledge graph would allow us to represent all relevant security knowledge – system components, threats, vulnerabilities, attackers, countermeasures, etc. – in one interconnected model. This model can encode multiple domains of knowledge. For example, it could simultaneously include an ontology of software architecture (services, data stores, data flows), an ontology of threats (like STRIDE categories or CAPEC attack patterns), and an ontology of controls (security controls, mitigations, requirements). The power of a graph is that these can all be linked: e.g., a node representing "SQL Injection" (from an attack pattern ontology) can be connected to a node representing "Customer Database" (from the system model) to indicate a threat, and that can connect to a node "Input Validation" (from a controls ontology) to indicate a mitigation. Because the graph is semantic, each of those node types has a definition and expected properties, enabling consistent structure and machine reasoning.

Dinis Cruz has often championed the view that "everything is really just graphs and maps," especially in security architecture. Through experimentation, it became evident that semantic knowledge graphs provide an effective solution for scaling the capture and analysis of complex security information [4]. In one project (MyFeeds.ai), large knowledge graphs were used to map relationships between entities of interest, but a key challenge was how to **scale** the graph's creation and curation [5]. This is where automation and AI come into play – by using LLMs or other techniques to help build and maintain the graph. A semantic graph approach addresses the earlier challenges by making threat model data structured, linked, and queryable. Rather than a Word document that sits on a shelf, the model becomes a living graph that tools and people can interact with. For instance:

- The graph structure enforces a level of consistency (you can define required relationships and attributes through the schema). This helps mitigate subjectivity; while humans still provide input, it's captured in a formal way. Over time, as more data populates the graph (from many models or sources), you build a **knowledge base** of threats and mitigations that can be reused.
- Graphs naturally break down silos. If each team contributes to the same enterprise knowledge graph (or at least uses common ontologies), their outputs become comparable and combinable. A vulnerability discovered in one application's threat model can be linked to a similar component in another application's model. Patterns emerge from the data – e.g., you might query the graph:

"show me all threat scenarios across all models that involve an outdated library" if that becomes a concern.
- Semantic graphs are inherently scalable in analyzing connectivity. Automated reasoning or queries can traverse the graph to find, say, all trust boundaries without an encryption control, or all critical assets not covered by a disaster recovery plan. This is far faster and more reliable than manually inspecting dozens of separate documents. The graph can also be updated incrementally; if a system changes or a new threat appears, you update that part of the graph, and all dependent analyses can instantly reflect the change.
- Perhaps most importantly, a semantic knowledge graph allows **contextual enrichment**. Because it is flexible in schema, you can plug in external data sources easily. For example, you could integrate an external **threat intelligence feed** or CVE database into the graph. If a new CVE emerges that affects a component in your system model, it can be linked to that component node, instantly flagging a relevant threat. Similarly, real-world incident data can be mapped onto the threat scenarios in the graph – if a certain attack has actually happened to your industry peers, that incident can be represented and tied to the scenarios in your model that correspond. Compliance frameworks (like ISO 27001 controls, or OWASP ASVS requirements) can be represented as another layer in the graph, ensuring that your threat mitigations cover required controls. All these linkages turn the threat model into a rich, **multi-dimensional map of risk**.

In summary, semantic knowledge graphs provide a *framework* to unify disparate pieces of security knowledge in a consistent, machine-readable form. By doing so, they enable a shift in threat modeling from a one-time subjective analysis to an ongoing data-driven activity. The next sections will describe how we propose to apply this concept to threat modeling, and what the architecture and technical implementation might look like.

Proposed Solution: Semantic Graph-Driven Threat Modeling

Our proposed solution is to implement threat modeling as a semantic knowledge graph that is continuously enriched and referenced throughout the software development and security lifecycle. This involves both a conceptual shift and a technical architecture. Conceptually, we treat the **threat model as a living graph** of interrelated security knowledge, rather than a static report. Technically, we need a system that can build, store, and query this graph effectively, integrating various sources of data.

Solution Overview and Key Features

At a high level, the approach works as follows:

1. **Define a Unified Schema/Ontologies:** We start by defining the ontologies (or schema) for our knowledge graph. This includes defining entity types such as *Asset* (e.g., a server, a microservice, a data store), *Threat* (an instance of a threat scenario, possibly linked to a taxonomy like STRIDE or CAPEC), *Vulnerability* (a weakness that could be exploited, possibly linked to CWE/CVE), *Mitigation/Control* (a safeguard or security control), *Actor* (could be an internal actor or external threat agent), and *Incident* (to capture real events). The schema also defines relationships: e.g., *Asset* "has" *Vulnerability*, *Threat* "targets" *Asset*, *Mitigation* "mitigates" *Threat*, *Incident* "is instance of" *Threat*, etc. By having a well-defined schema with semantic meaning, we ensure that when data is added to the graph, it adheres to a structure that tools can interpret. This addresses the need for consistency and determinism – the graph data conforms to known types and relations rather than arbitrary text. (Notably, in early prototypes one might let AI fill in relationships freely, but moving to an explicit schema is crucial for reliability [6].)

2. **Graph Construction Pipeline:** Next, we establish an automated pipeline to construct and update the graph. Multiple data sources feed into this pipeline:

3. **Architecture and Code**: Information about the system (assets, data flows, components) can be extracted from existing sources. This could be code (for instance, using static analysis or infrastructure-as-code to identify components and their interactions), system documentation, or diagrams. Generative AI can assist here by parsing natural language documentation or reading code repositories to identify key elements. For example, an LLM could be prompted with a microservice's README or API specification and produce a list of assets and trust boundaries, which are then added as nodes and edges in the graph. In prior work, a multi-phase LLM pipeline was successfully used to extract entities and relationships from text and convert them into graph form [7] [8]. We leverage similar techniques: the first phase might extract raw entities (components, data items, etc.), later phases might classify threats on those entities and map mitigations. The key is that the LLM output isn't free-form text but structured data (e.g., JSON) that matches our schema [9]. This yields typed objects which can be directly turned into graph nodes/edges.
4. **Threat Knowledge Bases**: We incorporate known threat intelligence – this could be in the form of a threat library, prior threat models, vulnerability databases, or compliance requirements. Rather than starting from a blank slate for each model, the system has a library of generic threats and mitigations (e.g., "SQL injection against a database" or "Data theft from S3 bucket without encryption"). These can be modeled as template subgraphs or patterns in the knowledge graph. When a new asset is added, the system can automatically link relevant threat nodes from this library to that asset (either through rules or AI suggestions). For instance, if the asset is a web application component, the system might attach the "OWASP Top 10" threat nodes to it by default, but only instantiate those that make sense (maybe based on characteristics of the asset, like if it handles authentication, then an "credential stuffing" threat might link).
5. **Real-World Data (Incidents, Advisories)**: The pipeline also monitors external data such as vulnerability advisories (CVE feeds), threat intel reports, or internally reported incidents. When new data arrives (say a CVE affecting a library in use, or an incident report of a breach in a similar industry), the graph is updated. For example, a CVE node gets linked to the asset (or a vulnerability node on that asset) if there is a match. An incident report might create a new *Incident* node that links to the Threat it exemplifies. Over time this builds a history of what has actually occurred, enriching the threat model with likelihood or impact context.

6. **Human Input and Review**: Security experts and developers are still in the loop – they can directly add nodes or relationships via an interface (which might be as simple as editing a YAML/JSON file or as visual as using a graph UI). The difference is those inputs go into the central graph rather than a separate doc. Also, during threat modeling workshops, instead of writing on a whiteboard, the team could populate the knowledge graph in real time (with the help of tools that create nodes/edges easily). The graph then serves as the living documentation.

7. **Semantic Enrichment and Inference:** Once data is in the graph, semantic enrichment can happen. Because our graph is based on ontologies, we can use **reasoners** or rule engines. For example, if a node is of type *WebApplication* and has property "usesAuthentication=false", we might have a rule that automatically creates a *Threat* node "Authentication Bypass" linked to that application (since lack of auth is a threat condition). Or a rule could infer risk level: if *Asset* is classified as "critical" and a linked *Threat* has no *Mitigation*, infer a high risk and flag it. In semantic web terms, one could use OWL constraints or SHACL rules to validate and flag issues in the graph data (like ensuring every threat on a critical asset has at least one mitigation – if not, that's a gap). The knowledge graph can also answer complex queries. For instance, one could query: "Find all threats that have an associated incident in the last 12 months and no

implemented mitigation; list the business process each threat impacts." This would traverse multiple parts of the graph (incidents, threats, mitigations, business context). Such queries give a very context-rich view for risk management, far beyond what static threat model documents offer. The use of semantic graphs thus **enables tracing and reasoning across layers of information**, making it easier to understand not just isolated threats but their full context. As noted in recent research, the semantic approach allows teams to trace the lineage of security decisions and understand the impact of changes in their threat models [10]. In our graph, one could literally follow a chain from a code change to an architecture node to a threat node to a mitigation and see how a change (like modifying an encryption library) might ripple through that chain.

8. **Integration with Development & Security Workflows:** A core principle is that the knowledge graph should not exist in a vacuum – it needs to integrate with the tools and processes engineers already use. This is in line with modern "DevSecOps" thinking that security should be embedded, not bolted on separately. Concretely, the threat modeling knowledge graph could be stored in a repository (e.g., as JSON or RDF files) alongside code. In fact, teams can maintain the graph data in Git, benefiting from version control and collaboration features [11]. Each update to the threat model (the graph) can be a commit, allowing traceability of how the model evolves over time with the system. Storing threat models in structured form in Git provides a historical record and ensures they are part of the development artifacts, not lost in emails or slide decks [11]. Moreover, the graph data can be linked to issue trackers: for instance, if a threat node is identified with no mitigation, an automatic ticket could be created linking to that node, effectively creating a workflow item for developers to address. Integration points also include CI/CD pipelines – e.g., a build can trigger a check of the threat graph against new code, or run queries to see if new components introduce new high-risk threats that need review. The result is that threat modeling becomes a continuous activity. Instead of a one-time meeting, the knowledge graph is continuously updated and consulted. This aligns with a vision of *continuous threat modeling*, where each code or design change prompts a security analysis update [12]. By embedding this into pipelines, we get nearer to real-time threat insights.

9. **Visualization and Layered Views:** While the data is in a graph database or files, it's crucial for humans to easily comprehend it. Visualization tools can be layered on top of the knowledge graph to allow interactive exploration. Users could toggle different "layers" of the graph: for example, a business view (showing business processes or high-level data flows and associated threats) versus a technical view (detailed software components and lower-level vulnerabilities). One might visualize just the subgraph of a particular microservice and its threats, or the connections between common threats across the entire enterprise. Graph visualization libraries or even Graphviz outputs can be used – Dinis Cruz's experiments highlight the importance of graph visualization for understanding the data [13]. By making the graphs first-class citizens, we enable new possibilities like graph-based dashboards for CISOs (showing risk hotspots in the architecture graph) or developer-friendly views integrated into IDEs (e.g., a plugin that shows the threat model graph relevant to the code file a developer is working on). The graph can also be queried via natural language using LLMs as a front-end – for instance, a developer could ask in plain English, "What are the top threats for the payment service and how are we mitigating them?" The system could translate that into a graph query and return the answer from the knowledge graph. This kind of accessibility will bring threat modeling knowledge to a wider audience beyond security specialists.

Putting it all together, the semantic graph-driven approach changes threat modeling from a static, subjective exercise to a dynamic, data-rich practice. The knowledge graph serves as a single source of truth for threats and mitigations, one that is **machine-readable and human-explainable**. By encoding

knowledge in the graph, we create a feedback loop: past models and external data inform new models, and new information updates the graph for all to benefit. The approach also ensures that **business and technical alignment** is achieved – because business context (like importance of an asset, or regulatory requirements) lives in the same graph as technical details, any analysis or report can draw on both. For example, an automatically generated threat modeling report could include a section on business impact ("this threat could lead to violation of GDPR compliance node X, which is linked to this asset") or on priority ("this component is high criticality, so its threats are ranked higher in risk"). This contrasts with today's situation where business impact and technical threats are often handled in separate processes.

Crucially, this solution leverages prior work and tools. The MyFeeds.ai project demonstrated a multi-phase LLM architecture to build semantic graphs from textual data, providing a blueprint for how unstructured input can yield structured graph output [14]. Open-source tools like **MGraph-DB** (also referred to as MGraph-AI, a memory-first graph database) are available to implement the graph storage and querying efficiently [15]. MGraph-DB was designed to be fast and serverless-friendly, using JSON as the storage format and keeping graphs in memory for quick AI-driven operations [16]. It supports semantic web concepts and can serve as the backbone for our threat modeling graph storage and manipulation [17]. By using such a graph database, we can easily version control the graph (storing as JSON), merge changes, and even perform type-safe operations in code. Additionally, frameworks for defining security ontologies (some academic works and standards exist for cybersecurity ontology) can bootstrap the schema definition. We envisage also utilizing OWASP's own projects – for instance, OWASP had initiatives like the OWASP Threat Dragon (which has a JSON threat model format) and there is mention of an OWASP SBot (semantic bot) which could be integrated [14]. These pieces indicate that the community is already moving towards more automation and data-centric approaches, and our proposal ties them together under the semantic graph paradigm.

Technical Implementation Considerations

Implementing a semantic knowledge graph for threat modeling involves a number of technical considerations and decisions. In this section, we delve into some of the key aspects, including the design of the schema/ontology, the choice of graph technology, the integration of multiple ontologies, the pipeline for data ingestion, and the use of inference engines. We also discuss how this system can integrate with existing source systems and maintain performance and security.

Schema and Ontology Design

Designing the schema is arguably the most foundational step. A well-designed ontology will determine how expressive and useful the threat knowledge graph will be. We recommend a modular ontology approach – breaking down the domain into sub-ontologies that can be linked. For example: - **System Model Ontology:** describes software and infrastructure components (e.g., classes for Application, Service, Database, DataFlow, TrustBoundary, etc.). It might inherit from a more generic IT ontology if available. - **Threat Ontology:** describes threats and weaknesses (e.g., Threat, Vulnerability, AttackTechnique, ThreatAgent). This could incorporate existing standards like STRIDE categories as an enumeration, or the MITRE ATT&CK tactics/techniques as an ontology of adversarial actions. There are also community knowledge bases (CAPEC for attack patterns, CWE for weaknesses) that can be mapped or imported. - **Mitigation/Control Ontology:** describes security controls and requirements (e.g., Control, Mitigation, SecurityRequirement). For instance, OWASP ASVS or NIST 800-53 controls could be represented here. - **Risk/Ops Ontology:** maybe for capturing risk ratings, incidents, and metrics (e.g., Incident, Likelihood, Impact, RiskAssessment).

These ontologies interlink. An Incident might be linked to a Threat (type of threat it realizes) and to an Asset (which asset was impacted). A Control might mitigate certain Vulnerabilities or satisfy certain Requirements. By keeping ontologies somewhat separate, we enable **multi-ontology support** – meaning we can plug in new taxonomies as needed without disrupting the core. For example, if a new threat framework becomes popular, we can add it as another set of classes and link them to existing ones (e.g., map a new framework's threat categories to STRIDE or to specific AttackTechnique nodes). This flexibility is important; it ensures the system can evolve and incorporate multiple standards simultaneously rather than forcing a single view.

We should use unique identifiers for every node (for instance, URIs if using RDF, or GUIDs in a property graph) to make referencing unambiguous. Adopting existing vocabularies where possible will save effort – e.g., using the vocabulary from an established cybersecurity ontology project. At the same time, we should be pragmatic: not every organization needs a full Semantic Web tech stack. We can implement the spirit of semantic graphs even with simpler technology (like JSON-based graphs in MGraph-DB) as long as the schema is clear and enforced.

One technical detail is ensuring the schema can be applied in a *type-safe* manner. The MGraph-AI project emphasizes type-safe classes for graph nodes [9], meaning when data is ingested, it's validated against class definitions. This can prevent errors (e.g., a Threat node missing a description field or an asset node missing a classification). By embedding such validation early (either through code or using formats like JSON Schema or SHACL for RDF), we maintain high data quality in the graph. High-quality data is crucial if we want to use inference reliably – garbage in, garbage out applies here.

Graph Database and Performance

Choosing the underlying graph database or storage solution is another key consideration. We have discussed MGraph-DB (Memory.DB) as a suitable choice given its design for speed and ease of use in AI scenarios. MGraph-DB keeps data in memory for fast operations and uses JSON for persistence [18], giving both performance and portability. It's also serverless-friendly and lightweight, which means it can be embedded in environments like CI pipelines or cloud functions without heavy infrastructure [19]. Moreover, it supports operations like merging graphs and filtering, which will be useful as we programmatically build and slice the threat models. The memory-first approach also aligns with the idea that during active threat modeling or analysis, we want the data readily available in memory for computations (like running an AI query or a large graph traversal) [20].

If an organization prefers a more traditional graph database, options include Neo4j (which is a popular property graph DB that also has some semantic capabilities via its APOC library or by storing RDF), or RDF triple stores like Apache Jena or GraphDB for full semantic web compliance. Neo4j could be used with the threat model schema defined as a property graph model, and Cypher queries could achieve a lot of the analysis. RDF stores would allow using SPARQL queries and OWL reasoning natively. The trade-offs typically involve performance vs. semantic richness – a property graph might be faster and easier for developers to work with, while an RDF/OWL graph might offer more powerful off-the-shelf reasoning capabilities. In practice, one could even use both: maintain a primary property graph but periodically export or sync data to an RDF store for certain analyses, if needed.

Versioning and collaboration are also crucial. Storing the graph in Git (or another VCS) means we need a serialization format. JSON or YAML work well for property graphs (e.g., listing nodes and edges). For RDF, Turtle or JSON-LD could be used. Each commit would ideally correspond to a logically consistent update (perhaps tied to a code change or a threat review meeting). With versioning, we can do diffs of threat models over time – e.g., see what changed between two releases in terms of threats and mitigations, which is extremely useful for audit and compliance. In fact, MGraph-AI explicitly lists

version control and easy diffs of graph data as a core requirement it addresses [21]. This means the team can review how a threat model has evolved just like reviewing code changes.

Data Ingestion and LLM Integration

Integrating LLMs into the pipeline warrants more discussion. Large Language Models can assist in two major ways: **information extraction** and **suggestion generation**. For information extraction, we can employ prompts that guide the LLM to output structured data about the system or threats. For example, given an architectural description, we prompt: "Extract all the assets (components, data stores) and trust boundaries described above, in JSON format with fields X, Y, Z." Thanks to the capability of modern LLMs to output JSON when instructed, and even use a provided JSON Schema, we can get a well-structured result [9]. Dinis's approach in MyFeeds.ai used OpenAI's structured output feature to ensure the LLM responses conformed to predefined Python classes, yielding typed JSON objects instead of free text [9]. We would similarly define classes for things like `Asset`, `ThreatScenario`, `Mitigation` and have the LLM populate them. It's important that the LLM operates in stages and with constraints; trying to have it "do everything in one go" often fails or produces irreproducible results [22]. Instead, we break the process: one LLM call to identify assets/flows, another to identify potential threats to those assets, another to suggest mitigations or link controls. Each step's output updates the graph. This multi-step design, as shown in MyFeeds.ai's 4-stage pipeline, brings determinism and clarity to the process [23]. We also incorporate human-in-the-loop where the AI's suggestions are reviewed before being finalized in the graph.

For suggestion generation, suppose we have a partial threat model graph built from known info – the LLM can be asked to fill gaps: "Given this asset and its attributes, list likely threat scenarios and affected assets." The advantage of a knowledge graph is that we can also feed the LLM a distilled context from the graph (e.g., relevant nodes and relationships) as part of the prompt, so it has the necessary domain knowledge to make suggestions. This is effectively using the graph as a contextual memory for the LLM, which can improve its accuracy in generating relevant threats or mitigations that fit the organization's context. Over time, one could fine-tune smaller models on the accumulated threat modeling data as well, creating specialized models that understand the ontology and can generate graph updates directly.

One challenge with LLMs is ensuring **provenance and explainability** of the generated content. Because we don't want to blindly trust AI outputs for something as sensitive as threat modeling, we need mechanisms to trace where a particular suggested threat came from. Semantic graphs can help here too – for instance, we could attach metadata to each node stating its source (e.g., "LLM suggestion based on OWASP Top10" or "added by user X on date Y"). If an LLM suggests a threat that references an external source (like a known attack), we can store a link to that source in the graph for justification. This aligns with the idea of provenance that Dinis has highlighted – connecting each statement to its original source [24]. Having such traceability is crucial to gain trust in an AI-augmented threat model.

Performance of the ingestion pipeline is also a consideration – parsing large codebases or documents via LLM could be time-consuming or costly. We can mitigate this by focusing on high-level architecture first (using summaries or system diagrams as input), and using specialized analysis tools for detailed code scanning rather than relying solely on LLMs. The knowledge graph can then be the place where outputs of various analysis tools converge (for example, a static code analyzer finds a data flow and it gets added as a graph edge; a dependency scanner finds a vulnerable library and it gets added as a vulnerability node linked to a component). The orchestration of these inputs can be done with scripts or an ETL (extract-transform-load) pipeline designed for the graph.

Inference and Analysis

With the graph populated, we want to leverage it for analysis. Two main approaches are rule-based inference and graph analytics: - **Rule-Based Inference:** Using semantic web rule engines or custom scripts, we can codify expert knowledge that automatically evaluates the graph. This could be done with OWL reasoning if our ontology is set up for it (e.g., we can declare that if an asset is of type `PublicFacing` and has data of type `PII`, then lack of encryption control implies a high risk – an OWL reasoner could detect a missing relationship and classify a risk). Alternatively, simpler if-then rules can be implemented in code to traverse the graph. For example, a Python script could find all threats without mitigations and mark them or raise alerts. The advantage of having data in a graph is that traversals are easy – e.g., starting from a critical asset node, you can programmatically walk all connected threat nodes and check their properties. This is far easier than parsing a document for the same info. We anticipate creating a library of common analysis rules which can be run as part of CI or on a schedule. These rules enforce completeness (e.g., if threat of type X exists, ensure control of type Y is present) and highlight conflicts or gaps. - **Graph Algorithms:** We can also use graph algorithms for deeper analysis. For instance, centrality algorithms might find which nodes (threats or assets) are most connected – perhaps highlighting a single point of failure that connects to many threats. Communities or clustering algorithms might identify groups of threats that often occur together, which could hint at an underlying systemic issue. Pathfinding could be used to simulate attack paths through a system: by linking vulnerabilities and exploits across components, an algorithm could find a path from an external attacker node to a crown jewel asset, which effectively is an end-to-end attack scenario. The knowledge graph is well-suited to this kind of analysis because it explicitly links everything.

The combination of these analyses can be used to produce dynamic reports or dashboards. For example, a "threat model report" can be generated by querying the graph and formatting results. Instead of writing these by hand for each project, a template could pull all threats for the project, group them by category, list mitigations, and even include cross-references to incidents or controls. Such generated documentation would be consistent across projects and always up-to-date (since it's just a view of the latest graph data) [25] [26]. Consistency in output is a known benefit of automation [27] – every threat model can have a thorough write-up even if human security experts are busy, because the AI and graph cover the basics. Of course, human review is still needed, but their effort shifts from manual writing to reviewing and refining the AI-generated content.

Integration with Source Systems

We touched on integration with development tools; here we expand on integrating with other source systems and data feeds: - **Issue Trackers and CMDBs:** Many organizations maintain a CMDB (configuration management database) or asset inventory. We can integrate that data as a starting point for the graph (e.g., auto-populate nodes for each server or service known in the inventory). Conversely, if the threat graph identifies an issue (like a missing control), it can create an issue in JIRA or similar to ensure it gets addressed. Bi-directional integration ensures the graph stays in sync with reality – e.g., closing a JIRA ticket for implementing a mitigation could trigger an update in the graph marking that threat as mitigated. - **Security Testing Tools:** Outputs from SAST (Static Application Security Testing), DAST (Dynamic Testing), vulnerability scanners, etc., can feed the graph. For instance, if SAST finds a potential SQL injection in code, that could be added as a *Vulnerability* node linked to the relevant *Asset* and *Threat* ("SQL injection"). This bridges the gap between design-time threat modeling and run-time testing – everything ends up as part of one knowledge base. Over time, correlating these could improve accuracy (if the graph knows a certain threat was actually proven by a pen-test, it gains weight in risk calculations). - **Incident Management and SIEM:** When incidents occur, analysts produce reports or tickets. By structuring and feeding these into the graph, you enrich the threat model with "ground truth" of what has happened. Each incident linking to a threat scenario can adjust likelihood

perceptions. Also, patterns of incidents can show where models might be missing something (if an incident doesn't map to any existing threat node, maybe the model needs an update). - **Compliance Management:** Many organizations use GRC (Governance, Risk, Compliance) tools to track controls and compliance requirements. We can import those as control nodes and requirement nodes in the graph. This ensures that when you perform threat modeling, you're also covering compliance bases. The graph can show, for example, which threats lack controls and also which compliance controls are unmet – if an auditor asks "are you covering control X", you can query the graph to show all relevant threats and mitigations tied to that control. - **External Threat Intel:** Feeds like STIX/TAXII (structured threat intelligence) could be ingested to create nodes about threat actors or campaigns, which might not be directly in a software threat model but could contextualize it. For example, if an APT group is known to target financial systems with a specific attack pattern, and your system is financial, linking that intel to your threat graph can prioritize those threats. In fact, forward-looking, one could imagine an industry-wide shared threat graph (perhaps hosted by a consortium or an open project) that companies contribute to. This could serve as a rich reference that individual organizations pull into their local threat models (filtered as needed). Semantic graphs are great for information sharing because of their standardized structure – one company's "Threat" node can be understood by another if they share ontology. This hints at "threat modeling disclosure frameworks" – akin to vulnerability disclosure but for sharing threat models. In the future, companies might share sanitized versions of their threat knowledge graphs or contribute to a common threat ontology to collectively advance the state of security knowledge.

Ensuring Determinism and Consistency

A known concern when introducing AI and automated data generation is maintaining deterministic, repeatable outputs. In our approach, we mitigate nondeterminism by: - Anchoring on a fixed schema (as discussed) – this forces outputs into a consistent form. - Using multiple smaller AI steps with human check-points, instead of one giant step. Each step's result is stored (and could be reviewed) before proceeding. - Incorporating feedback loops: if an AI suggestion is found wrong or irrelevant, that feedback can be encoded so the system doesn't repeat it. For example, if a particular threat keeps being suggested where it doesn't apply, we add a rule or tweak prompt to correct that. - Version control: We can always roll back or compare changes if something off happens, making debugging easier. - Provenance tagging: Every node could carry info on when and how it was generated (e.g., which prompt, which source). This transparency builds trust and allows troubleshooting of the model generation process.

The goal is for the threat modeling graph to produce the same core results given the same input context, thus being reliable enough for use in audits and planning. Achieving fully deterministic AI output is an ongoing area of development, but by constraining the problem and using structured output techniques, we get close to it, as evidenced in the MyFeeds.ai MVP where the LLM outputs were deterministic JSON structures for given inputs [9] .

Security Considerations of the Graph Approach

Storing all this information in one place – one might ask, does this create a new attractive target for attackers (i.e., a knowledge graph that, if compromised, details all your weaknesses)? It's a valid concern. We must treat the threat modeling knowledge graph with high confidentiality. That might mean hosting it on secure internal infrastructure, applying access controls (only authorized security/dev personnel can view it), and possibly partitioning highly sensitive info (maybe some extremely sensitive scenarios are kept in a more restricted part of the graph or abstracted). When using LLMs, care must be taken not to leak proprietary details in prompts to external services. Solutions include using on-premises LLMs or privacy-preserving techniques if using cloud APIs (some organizations use encryption

or redaction on prompt data). The white paper would not be complete without acknowledging that as we embrace AI and graphs, we must also guard against **AI-specific threats** – e.g., an attacker might attempt prompt injection on a system that uses an LLM to manipulate the threat model or hide certain threats [28]. The implementation must therefore include validation on AI outputs and not blindly trust them. Essentially, the security of the threat modeling platform itself becomes important – it should be threat-modeled too!

Thankfully, the transparency of a knowledge graph can help here: since all suggestions and changes are logged and visible, it might be easier to detect if something anomalous was introduced, compared to a scenario where an AI writes a natural language report that might hide subtle manipulations. By keeping humans in the loop and maintaining visibility, we can mitigate the risk of malicious interference.

In conclusion of this technical section, building a semantic knowledge graph for threat modeling is feasible with today's technology. Tools and methods from both the AI and semantic web worlds are converging to enable this: we have lightweight graph databases, powerful language models, and established ontologies. The key is integrating them thoughtfully to serve the needs of threat modeling. Next, we look ahead at what this approach unlocks for the future of the discipline.

Future Outlook

The use of semantic knowledge graphs in threat modeling is not just an incremental improvement – it could herald a paradigm shift in how organizations approach security design. In this final section, we explore the future possibilities and implications of widespread adoption of this approach, including how it transforms practices, what new opportunities it creates, and what challenges we must be prepared to address.

Toward Continuous, AI-Enhanced Threat Modeling: In the future, threat modeling is likely to become a continuous service integrated into the development pipeline, rather than a separate sporadic activity. With a semantic graph and AI, we can envision a **continuous threat modeling** process where every code commit, every infrastructure change, and every emerging threat intelligence update triggers an update to the threat model. Security analysis will be on-going and instantaneous, giving developers immediate feedback. For example, a developer saving a new microservice design could automatically receive a report of potential threats and mitigation suggestions within minutes, derived from the knowledge graph and AI reasoning. This aligns with the concept of treating threat modeling as a "living" part of DevSecOps. In fact, we might see AI assistants (perhaps integrated into IDEs or code review tools) that utilize the semantic threat graph to warn developers in real-time – much like spellcheck for security design. This will greatly reduce the window of exposure for vulnerabilities, as issues are caught and addressed at design or implementation time. It will also instill a security mindset in engineering culture, since the feedback is continuous and educational.

Scaling to Enterprise and Industry Knowledge Sharing: As organizations adopt semantic threat graphs, there is a big opportunity for standardization and knowledge sharing. Industry groups or communities (like OWASP, or sectors such as finance, healthcare) could maintain **shared ontologies and threat libraries** that companies contribute to. We already see the desire to make security knowledge more accessible and interconnected [29] [30]. OWASP, for instance, could transform its plethora of guides and cheat-sheets into a semantic knowledge base that plugs into these threat modeling graphs [31]. In the future, when a new vulnerability is discovered or a new mitigation technique is developed, updating a central knowledge graph could propagate that knowledge to all subscribing organizations' local graphs automatically. This network effect can drastically improve the industry's response to emerging threats. We might get to a point where publishing your threat model

(in an abstracted form) becomes a norm – much like open vulnerability disclosure. Organizations could share anonymized portions of their threat graph (e.g., patterns, mitigations that worked, etc.) contributing to collective security wisdom. Of course, this requires careful handling of sensitive info, but frameworks might emerge to facilitate this safely.

Rich Context = Better Decision Making: With context-rich graphs, executives and risk managers will have better information to make decisions. Instead of high-level, vague risk matrices, they could query the actual graph: e.g., "How many critical threats do we have open across all systems that lack mitigation and what's the potential impact in dollar terms?" If the graph is populated with the right data, the answer could be generated in seconds, with traceability down to each threat scenario and technical detail. This helps bridge the gap between technical findings and business impact – a perennial challenge in security. By aligning business and technical nodes in the graph, as we've described, the CISO can get a bird's-eye view that is backed by ground truth. This also aids in compliance and reporting. Generating evidence for controls or reports for audits could be done by extracting from the graph (which is much less effort than manually gathering data from various teams). In the future, regulators or standards bodies might even accept machine-readable submissions of security posture (for example, a regulator might query a company's knowledge graph or expect certain structures to be present as proof of diligence). This is speculative, but it stems from the increased structure and rigor that semantic threat modeling provides.

Enhanced Automation and "What-If" Analysis: Another exciting possibility is using the knowledge graph to perform simulations. Since the graph formalizes the system and threats, one could use automated reasoning to play "what-if" games. For instance, using the graph, generate all possible attack paths an attacker could take (within some bounds) and see which controls break those paths. This is like doing adversary emulation on the design level. An AI could simulate an attacker's perspective by traversing the graph from an attacker node to target nodes, revealing multi-step exploits that might not be obvious in a linear threat enumeration. Security teams could then proactively harden systems by cutting off these paths (adding controls or altering design). As AI models advance, they might be able to interface with the graph to propose and even implement mitigations. Imagine an AI ops agent that, upon detecting a high-risk gap in the threat graph, can open a pull request to implement a guardrail or apply a cloud configuration fix, then update the graph marking the threat as mitigated. This level of automation could drastically reduce the time from threat identification to remediation (something hinted at by the desire to shorten "discovery to mitigation" timeframes [32]).

Challenges and Considerations: With these opportunities come new challenges. One major challenge is **validation and trust**. As we rely on AI and graphs, we must ensure the outputs are correct and not missing critical things. Therefore, a lot of future work will likely focus on verification of AI-generated threat models – possibly using techniques like test cases for threat models (e.g., seeding known issues and seeing if the system catches them) or cross-validation by independent models. There is also the matter of **explainability**: stakeholders will demand to know why a certain threat was flagged as high risk by the AI. Our semantic approach aids in explainability, since the graph provides a chain of reasoning (one can navigate from a risk node to see all contributing factors). However, we might need to develop user-friendly explanation interfaces. Another challenge is **data security and privacy** as mentioned – especially if using external AI services or sharing knowledge across organizations. Techniques like federated learning or confidential computing may play a role in allowing benefits of shared intelligence without exposing raw data.

The skills required for threat modeling may also evolve. Future security engineers might need to be part-"knowledge engineers" – comfortable with ontologies, graph queries, and supervising AI. This is not as daunting as it sounds; graph and AI tooling is becoming more user-friendly, and many engineers might welcome the reduced grunt work. It does mean training and awareness need to catch up. The

community (including OWASP, industry groups, academia) should invest in developing best practices and education around semantic threat modeling. Initiatives like the OWASP integration of GenAI and graphs [29] [33] show that we're heading in this direction.

Visualization and User Experience in the Future: We expect significant improvements in how these threat knowledge graphs are visualized and interacted with. Perhaps AR/VR interfaces could allow teams to literally see their system's threat model in 3D graphs around them, making threat modeling sessions more engaging and intuitive. Or simpler, web-based interfaces will allow anyone to query the graph with natural language and see results with highlighted paths and nodes (some early examples of chatbots over security graphs are already appearing). If a developer can converse with the threat model ("Show me how this new feature could be attacked" and get an answer with a subgraph), it lowers the barrier to entry for continuous threat thinking.

Conclusion of Future Vision: In summary, advancing threat modeling with semantic knowledge graphs positions us to handle the growing complexity and pace of technology. It turns threat modeling into a **collective, ongoing intelligence activity** rather than an isolated paperwork drill. By combining semantic graphs, AI, and human expertise, we get the best of all worlds: the breadth and consistency of machine analysis with the insight and contextual judgment of seasoned professionals. The outcome is a threat modeling practice that is not only more effective at managing risk, but also more inclusive (bringing in developers, ops, and even execs into the security conversation via accessible knowledge interfaces) [34]. It aligns security with the business by embedding security knowledge into the fabric of organizational data.

The journey to get there is just beginning. Early experiments and research – including the ones by Dinis Cruz with GenAI and graphs, and industry efforts to scale threat modeling – are validating the concepts. This white paper aimed to bring these threads together into a coherent blueprint for the community. The authors, Dinis Cruz and ChatGPT Deep Research, encourage practitioners to pilot these ideas: start capturing threat models in graph form, leverage available tools like MGraph-DB, experiment with AI extraction of threats, and share your findings. By collaborating and iterating, the threat modeling community can build a future where we are not constrained by subjectivity and silos, but instead empowered by a **semantic, knowledge-driven approach** that keeps pace with innovation while solidifying security foundations.

---

[1] [34] Threat modelling at the speed of git
https://xntrik.wtf/aisa2024/

[2] [3] 5 Key Challenges for Threat Modelling in 2023
https://www.packetlabs.net/posts/key-challenges-for-threat-modelling-in-2023/

[4] [5] [6] [7] [8] [9] [13] [22] [23] [24] Building Semantic Knowledge Graphs with LLMs: Inside MyFeeds.ai's Multi-Phase Architecture
https://www.linkedin.com/pulse/building-semantic-knowledge-graphs-llms-inside-myfeedsais-dinis-cruz-jub5e

[10] [11] [12] [25] [26] [27] [28] [32] Threat Modeling Insider - February 2025 - Toreon
https://www.toreon.com/threat-modeling-insider-february-2025/

[14] [15] [29] [30] [31] [33] Semantic OWASP | Dinis Cruz
https://www.linkedin.com/posts/diniscruz_semantic-owasp-activity-7313492199692759042-zhy3

[16] [17] [18] [19] [20] [21] Introducing: MGraph-AI - A Memory-First Graph Database for GenAI and Serverless Apps
https://www.linkedin.com/pulse/introducing-mgraph-ai-memory-first-graph-database-genai-dinis-cruz-wxmde